Fitness Tracker Connected to the Internet Using a Wi-Fi Network

Frane Krapić*, Mladen Tomić*
* Faculty of Engineering, University of Rijeka, Rijeka, Croatia
krapicf@gmail.com

***Abstract*—Many open-source fitness trackers rely on Bluetooth Low Energy and a smartphone gateway, which limits its standalone operation and complicates continuous data upload. This paper presents a wearable activity tracker prototype that connects directly to Wi-Fi 6 and securely publishes activity metrics to a remote service using MQTT over TLS. The system is built on the Nordic nRF7002DK platform, combining an nRF5340 dual-core microcontroller with an nRF7002 Wi-Fi 6 radio, and it runs Zephyr RTOS via the nRF Connect SDK. Motion is captured with an MMA7361L three-axis analog accelerometer and processed on-device using a step detector with high-pass filtering, dynamic threshold adaptation, and simple activity classification from cadence and estimated speed. A round TFT display with capacitive touch provides a local user interface implemented using the Light and Versatile Graphics Library (LVGL). End-to-end functionality is validated by monitoring encrypted MQTT messages with standard Mosquitto client tools and by exercising a bidirectional control topic for basic commands. The prototype demonstrates a modular architecture for phone-independent wearable telemetry and provides a baseline for future work on energy optimization and improved motion analysis.**

***Keywords*—embedded systems, wearable devices, Wi-Fi 6, Zephyr RTOS, MQTT, TLS, LVGL, accelerometer**

I. Introduction and Related Work

Wearable fitness trackers are widely used for everyday activity monitoring, yet most existing devices still depend on a smartphone for network connectivity. In such systems, the wearable typically uses Bluetooth Low Energy (BLE) and offloads storage, visualization, and Internet access to a companion mobile application. While this architecture is power efficient, it introduces practical limitations: the user must keep a phone nearby, background connectivity can be interrupted by mobile operating-system power management, and the communication chain becomes more complex when troubleshooting reliability and security.

This work explores a phone-independent alternative: a wearable prototype with direct Wi-Fi 6 connectivity (IEEE 802.11ax) that uploads telemetry autonomously. Related implementations typically focus on BLE connectivity, phone applications, and cloud dashboards, or they use Wi-Fi-enabled microcontrollers but reduce security requirements and omit local user interaction. In contrast, this work integrates secure MQTT over TLS, a touch-enabled local GUI, and an on-device motion-processing pipeline on a Zephyr RTOS-based platform. The goal is not to maximize motion-analysis accuracy, but to demonstrate a complete and reproducible end-to-end system integration that others can replicate and extend.

A smartphone could in principle provide Wi-Fi connectivity and basic sensing, but it cannot serve as a miniaturized wrist-worn device. Beyond form factor, mobile operating systems impose background-process restrictions that interrupt continuous data collection, and they do not expose low-level Wi-Fi 6 features such as Target Wake Time to application developers. A purpose-built embedded platform avoids these constraints, provides full control over the sensing and networking stack, and is better suited to deployment contexts such as gyms, sports facilities, or team training grounds — where coaching staff can monitor athlete activity metrics continuously through shared Wi-Fi infrastructure without relying on each athlete's personal phone.

The main contributions are:

- An architectural and implementation proof-of-concept for a self-contained activity tracker with direct Wi-Fi 6 telemetry, validated in a controlled indoor environment.
- An embedded implementation on nRF7002DK using Zephyr RTOS, integrating analog accelerometer sampling, local GUI rendering, and MQTT networking.
- A secure MQTT communication setup (TLS, port 8883, keep-alive control) and a bidirectional command topic.
- End-to-end functional validation using standard Mosquitto tools during walking trials.
- A publicly available open-source firmware implementation as a reproducible reference.

The remainder of this section provides brief background and related work on Wi-Fi 6, BLE wearables, and open-source wearable projects. Section II describes the system design, Section III the implementation, Section IV the experimental setup, Section V the results and discussion, and Section VI concludes the paper.

A. Wi-Fi 6 for IoT-Class Devices

Wi-Fi 6 (IEEE 802.11ax) introduces OFDMA and MU-MIMO scheduling, spatial reuse mechanisms, and Target Wake Time (TWT) to improve spectral efficiency and allow stations to negotiate scheduled wake periods [1]. TWT lets a station and its access point agree on specific times at which the station will wake to exchange data; between

these scheduled intervals the radio can be fully powered down rather than periodically listening for beacons, which significantly reduces idle energy consumption for devices that transmit infrequently. These mechanisms are especially relevant for IoT devices that need predictable access and reduced idle listening. The nRF7002DK integrates the nRF7002 Wi-Fi 6 companion IC with the nRF5340 host MCU, and the reference Zephyr board design connects the radio over QSPI [2]. This split-radio architecture keeps the Wi-Fi MAC/PHY in the companion device while the host executes the application and networking logic.

B. BLE Wearable Ecosystems

BLE uses short-range, low-power links with 40 channels (three dedicated to advertising) for discovery and connection setup [3]. In typical wearable ecosystems, the device acts as a BLE peripheral while a smartphone aggregates data and provides Internet backhaul [4]. Connection intervals range from 7.5 ms to 4 s, trading latency for lower power consumption [5]. This architecture suits battery-constrained devices, but it ties cloud connectivity to the availability of a companion phone. A Wi-Fi-enabled wearable holds its own IP connection, removing this intermediary and simplifying the data path at the cost of higher radio power, a trade-off discussed in Section V.

C. Open-Source Wearable Projects

Several community-driven open-source wearable projects illustrate the current state of the art. ZSWatch [6] is a Zephyr RTOS-based smartwatch built on the nRF52 series that uses LVGL for its graphical interface but relies exclusively on BLE for connectivity, requiring a paired smartphone for any Internet access. InfiniTime [7], the default firmware for the PineTime smartwatch, runs FreeRTOS on an nRF52832 and similarly depends on a companion phone application for network backhaul. Both projects are mature, actively maintained platforms with real user communities, yet neither supports standalone Wi-Fi connectivity or direct cloud communication. Rather than proposing new algorithms or protocols, this work combines existing techniques such as accelerometer peak detection [8], MQTT messaging [9], and TLS-secured transport [10] into a wearable architecture that can operate without the companion smartphone required by ZSWatch and InfiniTime.

II. System Design

A. Hardware Architecture

The prototype is built on the Nordic nRF7002DK development kit [11]. The kit uses the dual-core Arm Cortex-M33 nRF5340 system-on-chip (SoC) as a host and integrates the nRF7002 Wi-Fi 6 companion radio for IP connectivity [2]. Motion sensing uses the MMA7361L, a MEMS three-axis analog-output accelerometer with selectable $\pm 1.5\,\mathrm{g}$ or $\pm 6\,\mathrm{g}$ sensitivity [12]. The sensor produces voltage signals proportional to acceleration on each axis; these are read directly by the MCU through three ADC channels and converted to acceleration values in software. The $\pm 1.5\,\mathrm{g}$ range was selected as suitable for detecting typical human walking and running motion. A 1.28-inch round TFT display (240×240 pixels) driven by the GC9A01 controller provides local visualization. The GC9A01 includes an integrated graphics RAM (GRAM) that stores the current frame, allowing partial screen updates without retransmitting the full image. A CST816D capacitive touch controller mounted over the display detects touch coordinates and gestures, reporting events to the MCU through I2C with an interrupt line for immediate notification. The display is driven over SPI for high-throughput pixel data transfer. The Wi-Fi companion radio is connected using a high-throughput serial interface (QSPI), enabling the host MCU to exchange network data with the radio [2].

TABLE I
Prototype components and interfaces

Component	Link	Purpose
MMA7361L accelerometer	ADC	3-axis motion sampling
GC9A01 TFT display	SPI	Local visualization
CST816D touch controller	I2C	Touch input events
nRF7002 Wi-Fi 6 radio	QSPI	Network connectivity

For data transport, the design adopts MQTT, a lightweight publish/subscribe messaging protocol suited to constrained devices and low-bandwidth links [9]. Performance evaluations comparing MQTT and CoAP show that protocol choice affects delay and bandwidth utilization depending on loss and congestion conditions [13], while security analysis shows that adding TLS increases latency and reduces throughput, requiring a balance between security and energy [14]. The system uses MQTT over TLS 1.3 [10] with a conservative keep-alive interval to secure telemetry without excessive overhead.

For motion processing, simple pedometer designs commonly apply time-domain peak detection on tri-axial accelerometer data, using windowed peak detection and thresholding to detect steps while rejecting noise [8]. The prototype extends this approach with a high-pass filter to remove gravity bias and a running standard deviation to set the detection threshold dynamically, combined with a 500 ms debounce interval.

The nRF7002DK was selected because it combines the nRF5340 host MCU and the nRF7002 Wi-Fi 6 companion IC on a single board with an established Zephyr board definition and full nRF Connect SDK support, enabling rapid subsystem integration without custom PCB development. As a development kit, however, it is larger than a consumer wearable and does not include battery management circuitry or a power-optimized layout. Miniaturization and battery-oriented hardware redesign are outside the scope of this work, as a compact wearable revision would require a different, more integrated platform and is therefore treated as a separate future effort rather than a direct extension of this prototype.

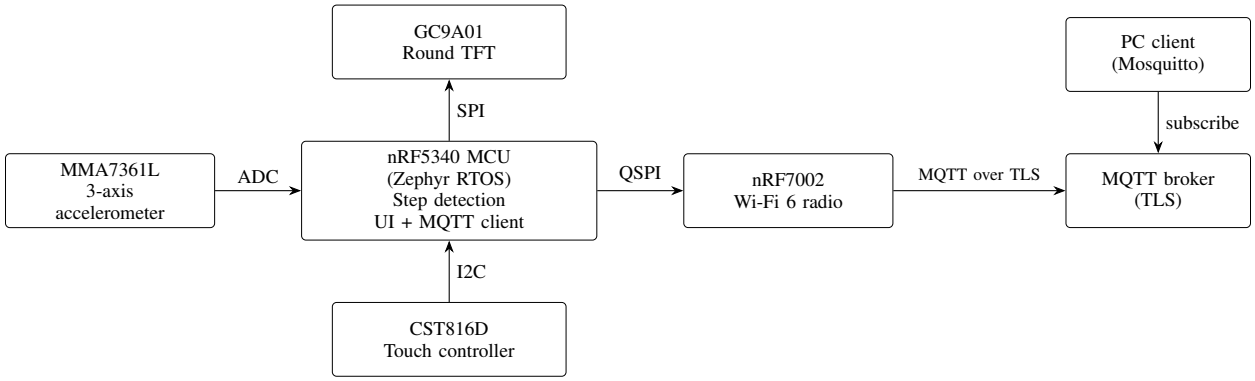


Fig. 1. System overview and data flow from sensing to secure MQTT telemetry.

B. Software Architecture and Data Flow

The software is built on Zephyr RTOS [15] using the nRF Connect SDK, with the application logic implemented in C. Zephyr provides a preemptive real-time scheduler, device-driver model, and integrated networking stack, which simplifies concurrency between sensing, UI updates, and MQTT communication [15]. The runtime functions are:

- Accelerometer sampling and step detection.
- GUI rendering and event handling via LVGL [16].
- Network connectivity and MQTT communication.

LVGL offers a lightweight widget framework and event-driven rendering suitable for microcontroller-based displays [16]. Fig. 1 shows the system-level data flow. Each detected step updates local metrics and triggers MQTT message generation. The MQTT client publishes telemetry as a JSON-formatted payload containing step count, distance, speed, calorie estimate, and activity level to a predefined topic, while a second topic is used to receive control commands (for example, resetting local counters or toggling publishing). If Wi-Fi or MQTT connectivity is temporarily unavailable, telemetry messages are queued in a fixed-size volatile RAM backlog (up to 32 entries) and retransmitted after the connection is restored. When the backlog is full the oldest entries are discarded so that the most recent state snapshots are preserved. Because the buffer resides entirely in RAM, queued data are lost on reset or power loss, and the mechanism is designed to handle short outages rather than serve as persistent logging. The LVGL-based GUI provides two screens:

- A main screen showing steps, distance, calorie estimate, speed, and activity level.
- A settings screen with spinbox controls for user height and weight and an apply button that recalculates stride parameters at runtime.

Touch events from the CST816D controller are mapped to LVGL callbacks, and displayed values are refreshed when a new step is detected.

III. Implementation

A. Platform and Configuration

Table II summarizes the key platform parameters and the development environment used for the prototype.

TABLE II
Platform parameters and development environment

Parameter	Value
MCU	nRF5340 (Arm Cortex-M33)
App core clock	128 MHz
App core Flash / RAM	1 MB / 512 KB
Wi-Fi companion IC	nRF7002 (802.11ax)
Host–radio interface	QSPI
SDK	nRF Connect SDK v3.x
RTOS	Zephyr RTOS
Toolchain	GNU Arm Embedded
Build system	CMake + west

For simplicity, Wi-Fi credentials (SSID and password), the broker hostname, and the MQTT topics are configured statically through the Zephyr project configuration. Since the prototype already includes a touch-enabled display, these parameters could be entered on-device through a setup screen in a production revision. MQTT communication is established using secure sockets and TLS, with the broker port set to 8883 and a keep-alive interval of 120 s. The implementation uses Zephyr's MQTT client APIs [17] and follows a clean-session connection setup. Example topic strings used in the prototype include wifi/fund/nrf/publish/fitness/topic1 for publishing and wifi/fund/nrf/subscribe/fitness/topic2 for subscribing.

B. Step Detection and Activity Estimation

After initializing ADC channels, the main loop continuously reads the three analog accelerometer axes and converts them into acceleration in units of g. From the three axis readings a_x, a_y, and a_z, the total acceleration magnitude is computed as:

$$m = \sqrt{a_x^2 + a_y^2 + a_z^2} \tag{1}$$

Gravity bias is then removed by a high-pass filter that subtracts a running mean $\bar{m}$ computed over the most recent 32 samples:

$$m_{\mathrm{hp}} = m - \bar{m} \tag{2}$$

A running standard deviation σ is computed over the most recent 64 samples and used to set a dynamic detection threshold:

$$t = \mu + k\,\sigma \tag{3}$$

where μ is the running mean of the high-pass-filtered signal and k is a tunable sensitivity factor. When σ falls below a no-motion gate (0.02 g), the detector is suppressed to avoid false triggers at rest. A step is registered on a rising-edge crossing of the dynamic threshold (3), with a minimum debounce interval of 500 ms between consecutive events. A warmup phase fills the filter windows before detection begins. The filter window sizes and sensitivity factor were tuned empirically for a single user during normal walking; their performance may degrade for different users, gait patterns, or activities such as stair climbing.

Cadence is derived from inter-step timing and smoothed with an exponential moving average (EMA, $\alpha = 0.25$). Step length is estimated from a model parameterized by user height: a base value proportional to height is blended with a cadence-dependent correction that uses separate coefficients for walking and running (transition at 150 steps/min). Distance accumulates from step length, and speed is computed from cadence and step length. Calorie expenditure is estimated using a simple model based on user weight, step count, and step length, providing a coarse energy indicator suitable for a fitness display. A basic activity level is produced using speed and cadence thresholds. Running is detected when speed exceeds approximately 0.9 m/s or cadence reaches approximately 150 steps/min, while the level returns to walking below approximately 0.7 m/s and 130 steps/min. These thresholds are intended for demonstration and are expected to vary across users and motion patterns. The step-length model has not been calibrated against a measured ground-truth reference across multiple subjects, so the reported distance and speed values remain approximate.

IV. Experimental Setup

The prototype was tested in a local Wi-Fi environment using a standard 802.11ax access point and a remote TLS-enabled MQTT broker on port 8883 with a 120 s keep-alive. Telemetry was captured by a separate Windows 10 PC that subscribed to the publish topic using Eclipse Mosquitto client tools (`mosquitto_sub`) [18]. The subscriber was configured with a CA certificate to verify the broker and establish encrypted communication. Indoor walking trials were conducted, and results were verified by visually cross-checking MQTT messages received on the PC against data shown on the on-device display.

The evaluation targeted the following functional aspects:

- Verifying that each detected step produced a corresponding MQTT message at the subscriber (end-to-end telemetry delivery).
- Confirming that the on-device display updated consistently with the published data (UI synchronization).
- Testing whether commands sent from the PC to the device subscribe topic were received and executed correctly, e.g., counter reset (bidirectional control).
- Ensuring that the TLS handshake completed successfully and that the broker rejected connections without a valid CA certificate (secure transport).

Step-counting accuracy was verified visually during walking trials by comparing the reported count against manually counted steps. The trial covered approximately 1000 steps on a flat indoor surface. After adjusting the height and weight parameters through the settings screen, step detection approached near-perfect accuracy on flat ground. Noticeably degraded accuracy was observed during stair climbing, consistent with the known limitations of threshold-based detectors noted in Section III-B. All telemetry messages published by the device were received at the PC subscriber with no observed drops during the indoor trial. The step-length model, parameterized by user height and stride factor, produced distance estimates with negligible deviation from the expected values in normal walking conditions.

V. Results and Discussion

Table III summarizes the functional tests performed and their outcomes. During walking trials, each step detection produced an MQTT message in real time, containing the current step count together with estimated distance, current speed, and activity level. The same events triggered local screen updates, enabling qualitative cross-checking between the on-device display and the received network messages. Bidirectional communication was confirmed by sending control messages from the PC to the device subscribe topic, enabling remote reset of local counters and changes in publish behavior. The TLS handshake completed without errors, and the broker correctly rejected connection attempts that omitted the CA certificate.

TABLE III
Functional validation results

Test scenario	**Result**	**Notes**
Step detection → MQTT publish	Pass	Message per step event
UI and MQTT data match	Pass	Visual cross-check
PC → device command (reset)	Pass	Counters reset confirmed
TLS handshake with CA cert	Pass	Port 8883, no errors
Connection without CA cert	Pass	Broker rejected as expected
Activity level switching	Pass	Walk/run levels observed

Beyond the functional results, Table IV contrasts the Wi-Fi 6 approach with conventional BLE-based trackers. Direct IP connectivity removes the companion-device requirement noted in Section I-B: the wearable uploads

data independently, avoiding synchronization issues and background-process restrictions inherent in BLE ecosystems [4]. In practice, the entire validation was conducted without a smartphone: telemetry was monitored directly via a Mosquitto client on a PC, demonstrating that the phone-independent data path is workable in a controlled environment. The trade-off is higher power consumption, since BLE is explicitly optimized for sleep-oriented operation with configurable connection intervals [3], [5], while Wi-Fi 6 features such as OFDMA and TWT can reduce idle power but require careful duty cycling to be effective [1]. Wi-Fi also ties the device to known access points, whereas a BLE tracker works wherever the user carries a paired phone. In summary, BLE favors battery life and universal portability, while Wi-Fi favors autonomous operation and direct cloud access. The proposed design accepts the power and coverage constraints as areas for future optimization through TWT scheduling and persistent on-device data buffering.

The validation presented here is functional and prototypical, conducted within a single controlled indoor Wi-Fi environment. TLS overhead on constrained devices has been characterized in prior work [14]; this prototype does not replicate those measurements. No controlled measurements of energy consumption, TLS handshake overhead, reconnection timing, or radio coverage were collected; these remain open items for future work. The volatile RAM buffer retains the most recent telemetry snapshots during outages and does not reconstruct a complete per-step history across connectivity interruptions.

TABLE IV
QUALITATIVE COMPARISON WITH BLE-BASED FITNESS TRACKERS

Aspect	**Wi-Fi 6 direct (this work)**	**BLE + phone hub**
Network path	Direct IP to MQTT broker over Wi-Fi	BLE peripheral; phone/hub provides Internet [4]
Power model	OFDMA/TWT features but requires duty cycling [1]	Designed for low-power with sleep and advertising [3]
Timing	TWT negotiation [1]	Connection interval (7.5 ms–4 s) [5]
Infra.	Requires Wi-Fi AP and provisioning	Requires nearby phone/hub; no Wi-Fi needed

VI. CONCLUSION

This paper presented a wearable activity tracker prototype that connects directly to a Wi-Fi 6 network and publishes encrypted telemetry without relying on a smartphone. Built on the Nordic nRF7002DK with Zephyr RTOS, the system combines on-device step detection, a touch-enabled LVGL interface, and secure MQTT communication into a single, self-contained platform. End-to-end functionality was validated during walking trials by cross-checking real-time MQTT messages against the on-device display and by exercising bidirectional control commands. The result is a reproducible reference implementation [19] that confirms the feasibility of phone-independent wearable telemetry in a controlled indoor environment. Future work will focus on quantifying energy consumption under controlled conditions, leveraging Wi-Fi 6 Target Wake Time scheduling to reduce idle power draw, refining motion-analysis models across multiple subjects, evaluating network coverage and memory constraints in real deployment scenarios, and exploring miniaturized hardware designs suitable for everyday carry.

REFERENCES

[1] E. Mozaffariahrar, F. Theoleyre, and M. Menth, "A Survey of Wi-Fi 6: Technologies, Advances, and Challenges," *Future Internet*, vol. 14, no. 10, Art. 293, Oct. 2022. Accessed: 2026-02-05. [Online]. Available: https://www.mdpi.com/1999-5903/14/10/293

[2] Zephyr Project, "nRF7002 DK — Zephyr Project Documentation," Accessed: 2026-02-05. [Online]. Available: https://docs.zephyrproject.org/latest/boards/nordic/nrf7002dk/doc/index.html

[3] E. Zanaj, G. Caso, L. De Nardis, A. Mohammadpour, O. Alay, and M.-G. Di Benedetto, "Energy Efficiency in Short and Wide-Area IoT Technologies—A Survey," *Technologies*, vol. 9, no. 1, Art. 22, Mar. 2021. Accessed: 2026-02-05. [Online]. Available: https://www.mdpi.com/2227-7080/9/1/22

[4] V. V. Tipparaju, K. R. Mallires, D. Wang, F. Tsow, and X. Xian, "Mitigation of Data Packet Loss in Bluetooth Low Energy-Based Wearable Healthcare Ecosystem," *Biosensors*, vol. 11, no. 10, Art. 350, Sep. 2021. Accessed: 2026-02-05. [Online]. Available: https://www.mdpi.com/2079-6374/11/10/350

[5] Silicon Labs, "Current Consumption," Bluetooth LE Fundamentals v6.0.0. Accessed: 2026-02-05. [Online]. Available: https://docs.silabs.com/bluetooth/6.0.0/bluetooth-fundamentals-system-performance/current-consumption

[6] J. Hammarberg, "ZSWatch — Open-source Zephyr-based smartwatch," GitHub repository. Accessed: 2026-02-05. [Online]. Available: https://github.com/jakkra/ZSWatch

[7] InfiniTime Contributors, "InfiniTime — Firmware for PineTime smartwatch," GitHub repository. Accessed: 2026-02-05. [Online]. Available: https://github.com/InfiniTimeOrg/InfiniTime

[8] Analog Devices, "AN-2554: Step Counting Using the ADXL367," Application Note, May 2023. Accessed: 2026-02-05. [Online]. Available: https://www.analog.com/en/resources/app-notes/an-2554.html

[9] OASIS, "MQTT Version 3.1.1," Approved: 2014-10-29. Accessed: 2026-02-05. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html

[10] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Aug. 2018. Accessed: 2026-02-05. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc8446

[11] Nordic Semiconductor, "nRF7002 DK — Development kit for the nRF7002 Wi-Fi 6 companion IC," Accessed: 2026-02-05. [Online]. Available: https://www.nordicsemi.com/Products/Development-hardware/nRF7002-DK

[12] NXP Semiconductors, "MMA7361L: ±1.5g, ±6g Three Axis Low-g Micromachined Accelerometer (Data Sheet)," Rev. 0, 2008. Accessed: 2026-02-05. [Online]. Available: https://www.nxp.com/docs/en/data-sheet/MMA7361L.pdf

[13] D. Thangavel, X. Ma, A. Valera, H.-X. Tan, and C. K.-Y. Tan, "Performance Evaluation of MQTT and CoAP via a Common Middleware," in *Proc. IEEE 9th Int. Conf. Intelligent Sensors, Sensor Networks and Information Processing (ISSNIP)*, Singapore, Apr. 2014, pp. 1–6. Accessed: 2026-02-05. doi:10.1109/ISSNIP.2014.6827678.

[14] Z. Laaroussi and O. Novo, "A Performance Analysis of the Security Communication in CoAP and MQTT," in *Proc. IEEE 18th Annu. Consumer Communications & Networking Conf. (CCNC)*, Jan. 2021, pp. 1–6. doi:10.1109/CCNC49032.2021.9369565. Accessed: 2026-02-05. [Online]. Available: https://research.aalto.fi/en/publications/a-performance-analysis-of-the-security-communication-in-coap-and-/

[15] Zephyr Project, "Zephyr Project," Accessed: 2026-02-05. [Online]. Available: https://www.zephyrproject.org/

[16] LVGL, "LVGL Documentation," Accessed: 2026-02-05. [Online]. Available: https://docs.lvgl.io/

[17] Zephyr Project, “MQTT — Zephyr Project Documentation,” Accessed: 2026-02-05. [Online]. Available: https://docs.zephyrproject.org/latest/connectivity/networking/api/mqtt.html
[18] Eclipse Foundation, “Eclipse Mosquitto,” Accessed: 2026-02-05. [Online]. Available: https://mosquitto.org/
[19] F. Krapić, “Fitness Tracker With Wi-Fi 6 Connectivity,” GitHub repository. Accessed: 2026-04-04. [Online]. Available: https://github.com/Krapic/Fitness-Tracker-With-Wifi6-Connectivity